AT1-C03

Y. HARIKA-192325023

Q1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.

Short Theory

Reinforcement Learning (RL) is a machine learning technique where an **agent learns by interacting with an environment**.

Components

- **Agent** → Learner or decision maker.
- **Environment** → World in which the agent operates.
- **State (S)** → Current situation of the environment.
- **Action (A)** → Decision taken by the agent.
- **Reward (R)** → Feedback received after taking an action.

The agent repeatedly:

1. Observes the current state.
2. Chooses an action.
3. Receives a reward.
4. Moves to the next state.
5. Learns to maximize the **cumulative reward**.

Cumulative Reward = Total rewards obtained over time.

CODE:

```
print("=== Reinforcement Learning Framework ===")

state = input("Enter Current State: ")
action = input("Enter Action Taken: ")
reward1 = int(input("Enter First Reward: "))
reward2 = int(input("Enter Second Reward: "))
next_state = input("Enter Next State: ")
cumulative_reward = reward1 + reward2
print("\n----- RL Interaction -----")
print("Current State :", state)
print("Action Taken  :", action)
print("Next State    :", next_state)
print("First Reward  :", reward1)
print("Second Reward :", reward2)
print("Cumulative Reward :", cumulative_reward)
if cumulative_reward > 0:
    print("Decision: Good policy (Positive reward)")
elif cumulative_reward == 0:
    print("Decision: Neutral policy")
else:
    print("Decision: Bad policy (Negative reward)")
```

```
Type "help", "copyright", "credits" or "license()" for more information.
>>> ================ RESTART: C:/Users/harik/Downloads/co-exp-1.py ================
=== Reinforcement Learning Framework ===
Enter Current State: start
Enter Action Taken:  Move Forward
Enter First Reward: 20
Enter Second Reward: 30
Enter Next State: Goal

----- RL Interaction -----
Current State : start
Action Taken  :  Move Forward
Next State    : Goal
First Reward  : 20
Second Reward : 30
Cumulative Reward : 50
Decision: Good policy (Positive reward)
>>>
```

Q2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

Short Theory

A **Markov Decision Process (MDP)** is a mathematical framework used in Reinforcement Learning to model sequential decision-making problems.

Components of MDP

- **State (S):** Current condition of the environment.
- **Action (A):** Choices available to the agent.
- **Transition Probability (P):** Probability of moving from one state to another after taking an action.
- **Reward Function (R):** Immediate reward received after an action.
- **Discount Factor (γ):** Determines the importance of future rewards ($0 \leq \gamma \leq 1$).

Python

```
print("=== Markov Decision Process (MDP) ===")

state = input("Enter Current State: ")
action = input("Enter Action: ")
next_state = input("Enter Next State: ")

transition_probability = float(input("Enter Transition Probability (0-1): "))
reward = float(input("Enter Reward: "))
gamma = float(input("Enter Discount Factor (0-1): "))

discounted_reward = reward * gamma

print("\n----- MDP Details -----")
print("Current State      :", state)
print("Action             :", action)
print("Next State         :", next_state)
print("Transition Probability :", transition_probability)
print("Reward             :", reward)
print("Discount Factor (γ) :", gamma)
print("Discounted Reward   :", discounted_reward)
```

```
Decision: Good policy (Positive reward)
>>>
============================================================
=== Markov Decision Process (MDP) ===
Enter Current State: S1
Enter Action: Left
Enter Next State: S3
Enter Transition Probability (0-1): 0.5
Enter Reward: 150
Enter Discount Factor (0-1): 0.3

----- MDP Details -----
Current State       : S1
Action              : Left
Next State          : S3
Transition Probability : 0.5
Reward              : 150.0
Discount Factor (γ) : 0.3
Discounted Reward   : 45.0
>>>
```

Q3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

Short Theory

In Reinforcement Learning (RL), the **Policy** determines the action an agent should take in a given state, while the **Value Function** estimates the expected future rewards.

Components

- **Policy (π):** A strategy that maps states to actions.
- **State-Value Function (V(s)):** Expected cumulative reward starting from state **s**.
- **Action-Value Function (Q(s,a)):** Expected cumulative reward after taking action **a** in state **s**.
- **Bellman Equation:** Updates the value of a state using the reward and the value of the next state.

Bellman Equation:

where:

- **R** = Immediate Reward
- **γ** = Discount Factor
- **V(s')** = Value of Next State

CODE:

```
print("=== Policy and Value Function ===")

state = input("Enter Current State: ")
action = input("Enter Action: ")

reward = float(input("Enter Immediate Reward: "))
gamma = float(input("Enter Discount Factor (0-1): "))
next_value = float(input("Enter Value of Next State: "))

# Bellman Equation
current_value = reward + (gamma * next_value)

print("\n----- RL Evaluation -----")
print("Current State :", state)
print("Action        :", action)
print("Reward        :", reward)
print("Discount γ    :", gamma)
print("Next State Value :", next_value)
print("Current State Value V(s):", current_value)
```

```
s/co-exp-1.py ====================================
================================
=== Policy and Value Function ===
Enter Current State: S1
Enter Action: LEFT
Enter Immediate Reward: 18
Enter Discount Factor (0-1): 0.2
Enter Value of Next State: 20

----- RL Evaluation -----
Current State : S1
Action        : LEFT
Reward        : 18.0
Discount γ    : 0.2
Next State Value : 20.0
Current State Value V(s): 22.0
>>>
```

Q4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ε-greedy and softmax approaches in balancing learning efficiency.

Short Theory

The **Exploration–Exploitation Trade-off** is an important concept in Reinforcement Learning.

- **Exploration:** The agent tries new actions to discover better rewards.
- **Exploitation:** The agent selects the best-known action to maximize reward.

A good RL agent balances both exploration and exploitation.

Strategies

1. ε-Greedy

- Chooses the best action with probability **1 – ε**.
- Chooses a random action with probability **ε**.
- Simple and widely used.

2. Softmax

- Assigns probabilities to actions based on their Q-values.
- Better actions have higher probability but all actions can still be selected.
- Provides smoother exploration than ε-Greedy.

CODE:

```
import random

print("=== Exploration vs Exploitation ===")

best_action = input("Enter Best Known Action: ")
epsilon = float(input("Enter Epsilon Value (0 to 1): "))

random_action = input("Enter Random Action: ")

r = random.random()

print("\nGenerated Random Number:", round(r, 2))

if r < epsilon:
    chosen_action = random_action
    strategy = "Exploration"
else:
    chosen_action = best_action
    strategy = "Exploitation"

print("\n----- Result -----")
```

```
print("Best Action :", best_action)
print("Random Action :", random_action)
print("Epsilon :", epsilon)
print("Selected Strategy :", strategy)
print("Chosen Action :", chosen_action)
```

```
>>>
    == RESTART: C:/Users/harik/Downloads/co-exp-1.py ==
    === Exploration vs Exploitation ===
    Enter Best Known Action: LEFT
    Enter Epsilon Value (0 to 1): 0.3
    Enter Random Action: MOVE RIGHT

    Generated Random Number: 0.36

    ----- Result -----
    Best Action : LEFT
    Random Action : MOVE RIGHT
    Epsilon : 0.3
    Selected Strategy : Exploitation
    Chosen Action : LEFT
>>>
```

Q5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies.

Short Theory

Reward Shaping is a technique in Reinforcement Learning where additional rewards or penalties are provided to guide the agent toward the desired behavior.

Advantages

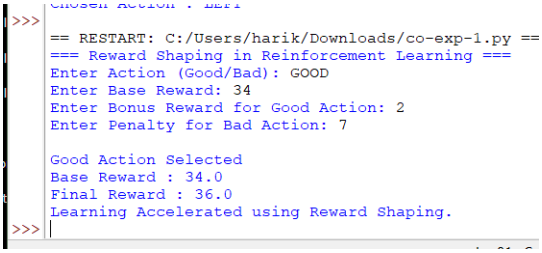
- Speeds up the learning process.
- Encourages the agent to take better actions.
- Helps avoid **suboptimal policies** (poor solutions).
- Improves convergence to the optimal policy.

Suboptimal Policy

A **suboptimal policy** is a policy that does not produce the maximum possible cumulative reward because the agent gets stuck choosing less effective actions.

CODE:

```
print("=== Reward Shaping in Reinforcement Learning ===")

action = input("Enter Action (Good/Bad): ")

base_reward = float(input("Enter Base Reward: "))
bonus_reward = float(input("Enter Bonus Reward for Good Action: "))
penalty = float(input("Enter Penalty for Bad Action: "))

if action.lower() == "good":
    final_reward = base_reward + bonus_reward
    print("\nGood Action Selected")
else:
    final_reward = base_reward - penalty
    print("\nBad Action Selected")

print("Base Reward :", base_reward)
print("Final Reward :", final_reward)

if final_reward > base_reward:
    print("Learning Accelerated using Reward Shaping.")
else:
    print("Agent may follow a Suboptimal Policy.")
```



Q6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Short Theory

Applying **Reinforcement Learning (RL)** in real-world robotics is challenging because robots interact with physical environments where mistakes can be costly.

Major Challenges

- **Safety:** The robot must avoid dangerous actions that could damage itself or its surroundings.
- **Sample Efficiency:** RL often requires many training episodes, which is time-consuming and expensive in robotics.
- **Environment Variability:** Changes in lighting, obstacles, terrain, or weather can affect the robot's performance.

These challenges influence the robot's ability to learn an optimal policy efficiently and safely.

CODE:

```
print("=== RL Challenges in Robotics ===")

robot_name = input("Enter Robot Name: ")

safety = int(input("Enter Safety Score (0-100): "))
sample_efficiency = int(input("Enter Sample Efficiency Score (0-100): "))
environment = int(input("Enter Environment Stability Score (0-100): "))

performance = (safety + sample_efficiency + environment) / 3

print("\n----- Robot Performance -----")
print("Robot Name :", robot_name)
print("Safety Score :", safety)
print("Sample Efficiency :", sample_efficiency)
print("Environment Stability :", environment)
print("Overall Performance :", round(performance, 2))

if performance >= 80:
   print("Performance: Excellent")
elif performance >= 60:
   print("Performance: Good")
```

```
elif performance >= 40:
   print("Performance: Average")
else:
   print("Performance: Poor")
```

```
>>>
   == RESTART: C:/Users/harik/Downloads/co-exp-1.py ==
   === RL Challenges in Robotics ===
   Enter Robot Name: ROBO
   Enter Safety Score (0-100): 60
   Enter Sample Efficiency Score (0-100): 70
   Enter Environment Stability Score (0-100): 80

   ----- Robot Performance -----
   Robot Name : ROBO
   Safety Score : 60
   Sample Efficiency : 70
   Environment Stability : 80
   Overall Performance : 70.0
   Performance: Good
>>>
```

Q7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

Short Theory

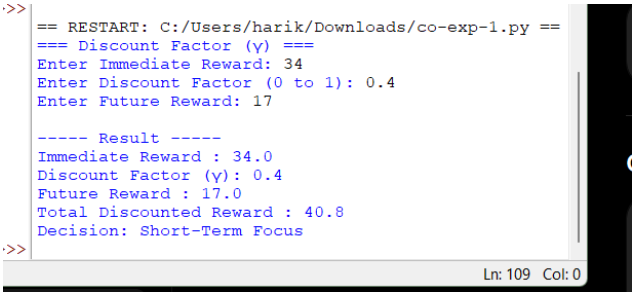
The **Discount Factor (γ)** is an important parameter in Reinforcement Learning that determines how much importance the agent gives to **future rewards** compared to **immediate rewards**.

- **γ = 0** → Agent considers only immediate rewards (short-term decisions).
- **γ close to 1** → Agent considers future rewards (long-term decisions).

In **episodic tasks**, the episode ends after reaching a goal or terminal state. The discount factor helps the agent maximize the total reward until the episode ends.

CODE:

```
print("=== Discount Factor (γ) ===")

reward = float(input("Enter Immediate Reward: "))
gamma = float(input("Enter Discount Factor (0 to 1): "))
future_reward = float(input("Enter Future Reward: "))

total_reward = reward + (gamma * future_reward)

print("\n----- Result -----")
print("Immediate Reward :", reward)
print("Discount Factor (γ):", gamma)
print("Future Reward :", future_reward)
print("Total Discounted Reward :", total_reward)

if gamma < 0.5:
   print("Decision: Short-Term Focus")
else:
   print("Decision: Long-Term Focus")
```



Q8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.

Short Theory

Reinforcement Learning (RL) algorithms are classified into **Model-Based RL** and **Model-Free RL**.

Model-Based RL

- Learns or knows the environment model (transition and reward functions).
- Uses planning before taking actions.
- Requires fewer interactions with the environment.
- Suitable for predictable environments.

Model-Free RL

- Does not know the environment model.
- Learns directly through trial and error.
- Simpler to implement.
- Suitable for dynamic and complex environments.

Comparison

Model-Based RL	**Model-Free RL**
Uses environment model	No environment model
Faster learning	Slower learning
Requires planning	Learns by experience
Sample efficient	Needs more training samples
Best for known environments	Best for unknown/dynamic environments

CODE:

```
# Model-Based vs Model-Free RL


print("=== Model-Based vs Model-Free RL ===")
```

```
algorithm = input("Enter RL Type (Model-Based/Model-Free): ")
environment = input("Enter Environment (Static/Dynamic): ")
episodes = int(input("Enter Number of Training Episodes: "))

print("\n----- Evaluation -----")
print("RL Type :", algorithm)
print("Environment :", environment)
print("Training Episodes :", episodes)

if algorithm.lower() == "model-based":
    print("Uses an environment model.")
    print("Fast learning with planning.")
    print("Best for predictable environments.")
else:
    print("Learns through trial and error.")
    print("No environment model required.")
    print("Suitable for dynamic environments.")

if environment.lower() == "dynamic":
    print("Recommendation: Model-Free RL performs better.")
else:
    print("Recommendation: Model-Based RL performs better.")
```

```
>>
== RESTART: C:/Users/harik/Downloads/co-exp-1.py ==
=== Model-Based vs Model-Free RL ===
Enter RL Type (Model-Based/Model-Free): MODEL-BASED
Enter Environment (Static/Dynamic): DYNAMIC
Enter Number of Training Episodes: 300

----- Evaluation -----
RL Type : MODEL-BASED
Environment : DYNAMIC
Training Episodes : 300
Uses an environment model.
Fast learning with planning.
Best for predictable environments.
Recommendation: Model-Free RL performs better.
>>
```

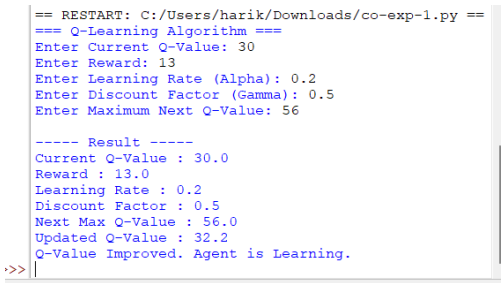
Q9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior.
Short Theory

Q-Learning is a **model-free Reinforcement Learning algorithm** that learns the optimal action-value function (**Q-value**) through trial and error.
The Q-value represents the expected reward for taking an action in a given state.
Q-Learning Update Equation
Where:

- **Q(s,a)** = Current Q-value
- **α (Alpha)** = Learning rate
- **R** = Immediate reward
- **γ (Gamma)** = Discount factor
- **max Q(s',a')** = Maximum Q-value of the next state

During training, the Q-values gradually converge to the optimal values.
CODE:

```
# Q-Learning Update Example

print("=== Q-Learning Algorithm ===")

current_q = float(input("Enter Current Q-Value: "))
reward = float(input("Enter Reward: "))
learning_rate = float(input("Enter Learning Rate (Alpha): "))
discount_factor = float(input("Enter Discount Factor (Gamma): "))
next_max_q = float(input("Enter Maximum Next Q-Value: "))

# Q-Learning Update Formula
new_q = current_q + learning_rate * (
    reward + discount_factor * next_max_q - current_q
)

print("\n----- Result -----")
print("Current Q-Value :", current_q)
print("Reward :", reward)
print("Learning Rate :", learning_rate)
print("Discount Factor :", discount_factor)
print("Next Max Q-Value :", next_max_q)
print("Updated Q-Value :", round(new_q, 2))

if new_q > current_q:
    print("Q-Value Improved. Agent is Learning.")
else:
    print("No Significant Improvement.")
```



Q10. Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.
Short Theory

In Reinforcement Learning, learning methods are classified into **On-Policy** and **Off-Policy** based on how the agent learns.

On-Policy Learning

- Learns using the **same policy** that is used to make decisions.
- Continuously updates the policy while interacting with the environment.
- Example: **SARSA (State-Action-Reward-State-Action)**

Off-Policy Learning

- Learns the optimal policy while following a different behavior policy.
- Can learn from previously collected experiences.
- Example: **Q-Learning**

Comparison

On-Policy	**Off-Policy**
Learns from current policy	Learns from another policy
Safer learning	Faster learning
Less sample efficient	More sample efficient
Example: SARSA	Example: Q-Learning

CODE:

```
print("=== On-Policy vs Off-Policy ===")

method = input("Enter Learning Method (On-Policy/Off-Policy): ")
algorithm = input("Enter Algorithm Name: ")
episodes = int(input("Enter Number of Episodes: "))

print("\n----- Learning Details -----")
print("Method :", method)
print("Algorithm :", algorithm)
print("Training Episodes :", episodes)

if method.lower() == "on-policy":
    print("\nCharacteristics:")
    print("- Learns using the current policy.")
    print("- Safer exploration.")
    print("- Example: SARSA")
    print("- Suitable for environments requiring safe learning.")
else:
    print("\nCharacteristics:")
    print("- Learns using a different policy.")
    print("- Faster convergence.")
    print("- Example: Q-Learning")
    print("- Suitable for large and dynamic environments.")
```

```
>>>
== RESTART: C:/Users/harik/Downloads/co-exp-1.py ==
=== On-Policy vs Off-Policy ===
Enter Learning Method (On-Policy/Off-Policy): OFF-P
OLICY
Enter Algorithm Name: Q-LEARNING
Enter Number of Episodes: 3000

----- Learning Details -----
Method : OFF-POLICY
Algorithm : Q-LEARNING
Training Episodes : 3000

Characteristics:
- Learns using a different policy.
- Faster convergence.
- Example: Q-Learning
- Suitable for large and dynamic environments.
>>>
```

Ln: 158 Col: